

1)

Um driver de dispositivo é basicamente um software específico, normalmente feito pelo fabricante do hardware, que serve para controlar aquele dispositivo. Ele é responsável por lidar com toda a parte complexa de baixo nível, como acessar registradores e tratar interrupções, escondendo esses detalhes do restante do sistema. A interface dele com o sistema operacional funciona através de um modelo padronizado, onde o SO define um conjunto de funções comuns que os drivers devem ter. Na prática, o sistema operacional usa uma tabela de ponteiros de funções para acessar o driver. Assim, quando o sistema precisa ler ou escrever algo, ele chama a função através dessa tabela sem precisar saber como o driver executa a tarefa internamente, o que permite instalar drivers novos sem ter que alterar o código do núcleo do sistema operacional.

2)

A diferença fundamental está no bloqueio do processo:

- **Síncrona (Bloqueante):** Quando um programa inicia uma chamada de E/S (como `read`), ele é suspenso (bloqueado) até que os dados estejam disponíveis no buffer. A CPU pode executar outros processos enquanto isso.
- **Assíncrona (Orientada à Interrupção):** O programa inicia a transferência e continua executando outras tarefas. A CPU ou o programa é interrompido posteriormente quando os dados chegam.

Qual é a mais conveniente? A comunicação síncrona é muito mais conveniente para programas de usuário. É muito mais fácil para um programador escrever e raciocinar sobre um código que segue uma sequência lógica ("leia isto, depois faça aquilo com o dado lido") do que escrever um código complexo que precisa lidar com eventos assíncronos e interrupções aleatórias. O sistema operacional faz o trabalho sujo de gerenciar a assincronia do hardware e fazê-la parecer síncrona para o usuário.

3)

O software de E/S no espaço do usuário consiste em bibliotecas e programas que executam fora do kernel do sistema operacional, facilitando a E/S ou gerenciando dispositivos dedicados. Ele é dividido principalmente em:

1. **Bibliotecas:** Rotinas ligadas aos programas do usuário (como `stdio` em C contendo `printf` e `write`). Elas preparam os dados, formatam a entrada/saída (por exemplo, convertendo binário para ASCII) e realizam as chamadas de sistema (syscalls) para o núcleo.
2. **Sistemas de Spooling:** Usados para dispositivos dedicados (como impressoras) em sistemas multiprogramados. Para evitar conflitos de acesso direto, um processo especial chamado daemon e um diretório de spool são usados. O usuário não envia dados direto para a impressora, mas sim para o diretório de spool; o daemon, que é o único com permissão para usar o dispositivo, pega os arquivos da fila e os imprime.

4)

Arquivo: common.h

```

#ifndef COMMON_H
#define COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

// Nomes para a memória compartilhada e semáforos
#define SHM_NAME "/exemplo_shm"
#define SEM_MUTEX "/sem_mutex"
#define SEM_EMPTY "/sem_empty"
#define SEM_FULL "/sem_full"

// Tamanho do buffer (quantos números cabem na memória)
#define BUFFER_SIZE 5

// Estrutura da memória compartilhada
typedef struct
{
    int buffer[BUFFER_SIZE];
    int in; // Índice de inserção
    int out; // Índice de remoção
} shared_data_t;

#endif

```

Arquivo: ex4_produtores.c

```

#include "common.h"

int main()
{
    // 1. Criar/Abrir objeto de memória compartilhada
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open");
        exit(1);
    }

    // 2. Definir o tamanho da memória
    ftruncate(shm_fd, sizeof(shared_data_t));

    // 3. Mapear na memória do processo
    shared_data_t *shared_mem = mmap(NULL, sizeof(shared_data_t),

```

```

PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
if (shared_mem == MAP_FAILED)
{
    perror("mmap");
    exit(1);
}

// Inicializar índices
shared_mem->in = 0;
shared_mem->out = 0;

// 4. Criar/Abrir Semáforos
sem_t *sem_mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
sem_t *sem_empty = sem_open(SEM_EMPTY, O_CREAT, 0666, BUFFER_SIZE);
sem_t *sem_full = sem_open(SEM_FULL, O_CREAT, 0666, 0);

srand(time(NULL));

printf("Produtor iniciado. Gerando números...\n");

while (1)
{
    int num = rand() % 100; // Gera número aleatório

    // Espera haver espaço vazio
    sem_wait(sem_empty);
    // Entra na região crítica
    sem_wait(sem_mutex);

    // Escreve na memória
    shared_mem->buffer[shared_mem->in] = num;
    printf("[Produtor] Escreveu: %d na posição %d\n", num, shared_mem-
>in);
    shared_mem->in = (shared_mem->in + 1) % BUFFER_SIZE;

    // Sai da região crítica
    sem_post(sem_mutex);
    // Sinaliza que há um item novo (full)
    sem_post(sem_full);

    sleep(1); // Unidade de tempo configurável
}

// Limpeza (inalcançável no loop infinito)
munmap(shared_mem, sizeof(shared_data_t));
close(shm_fd);
return 0;
}

```

```

#include "common.h"

int main()
{
    // 1. Abrir objeto de memória compartilhada (já criado pelo produtor)
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open (Execute o produtor primeiro)");
        exit(1);
    }

    // 2. Mapear na memória
    shared_data_t *shared_mem = mmap(NULL, sizeof(shared_data_t),
                                     PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);
    if (shared_mem == MAP_FAILED)
    {
        perror("mmap");
        exit(1);
    }

    // 3. Abrir Semáforos existentes
    sem_t *sem_mutex = sem_open(SEM_MUTEX, 0);
    sem_t *sem_empty = sem_open(SEM_EMPTY, 0);
    sem_t *sem_full = sem_open(SEM_FULL, 0);

    printf("Consumidor iniciado. Aguardando dados...\n");

    while (1)
    {
        // Espera haver algo para consumir
        sem_wait(sem_full);
        // Entra na região crítica
        sem_wait(sem_mutex);

        // Lê da memória
        int num = shared_mem->buffer[shared_mem->out];
        printf("[Consumidor] Leu: %d da posição %d\n", num, shared_mem-
>out);
        shared_mem->out = (shared_mem->out + 1) % BUFFER_SIZE;

        // Sai da região crítica
        sem_post(sem_mutex);
        // Sinaliza que há um espaço vazio
        sem_post(sem_empty);

        // Simula tempo de processamento
        sleep(2);
    }
    return 0;
}

```

5)

Arquivo: ex5_produtores.c

```
/* produtor_mprotect.c */
#include "common.h"

int main()
{
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open");
        exit(1);
    }

    ftruncate(shm_fd, sizeof(shared_data_t));

    // --- MUDANÇA EXERCÍCIO 5 ---
    // 1. Inicialmente mapeia com PROT_NONE (sem acesso)
    printf("Mapeando memória com PROT_NONE...\n");
    shared_data_t *shared_mem = mmap(NULL, sizeof(shared_data_t),
                                      PROT_NONE, MAP_SHARED, shm_fd, 0);

    if (shared_mem == MAP_FAILED)
    {
        perror("mmap");
        exit(1);
    }

    // Se tentássemos acessar shared_mem aqui, ocorreria um Segmentation
    // Fault.

    printf("Alterando permissões com mprotect para READ|WRITE...\n");

    // 2. Usar mprotect para permitir acesso
    if (mprotect(shared_mem, sizeof(shared_data_t), PROT_READ | PROT_WRITE)
    == -1)
    {
        perror("mprotect");
        exit(1);
    }
    // -----

    // O restante do código segue idêntico ao exercício 4...
    shared_mem->in = 0;
    shared_mem->out = 0;

    sem_t *sem_mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
    sem_t *sem_empty = sem_open(SEM_EMPTY, O_CREAT, 0666, BUFFER_SIZE);
    sem_t *sem_full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
```

```

srand(time(NULL));

printf("Produtor iniciado (memória protegida e liberada).\n");

while (1)
{
    int num = rand() % 100;
    sem_wait(sem_empty);
    sem_wait(sem_mutex);

    shared_mem->buffer[shared_mem->in] = num;
    printf("[Produtor] Escreveu: %d\n", num);
    shared_mem->in = (shared_mem->in + 1) % BUFFER_SIZE;

    sem_post(sem_mutex);
    sem_post(sem_full);
    sleep(1);
}
return 0;
}

```

Arquivo: ex5_consumidor.c

```

#include "common.h"

int main()
{
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd == -1)
    {
        perror("shm_open (Execute o produtor primeiro)");
        exit(1);
    }

    // --- MUDANÇA EXERCÍCIO 5 ---
    // 1. Mapeia inicialmente SEM permissão (PROT_NONE)
    printf("Consumidor: Mapeando memória com PROT_NONE...\n");
    shared_data_t *shared_mem = mmap(NULL, sizeof(shared_data_t),
                                     PROT_NONE, MAP_SHARED, shm_fd, 0);
    if (shared_mem == MAP_FAILED)
    {
        perror("mmap");
        exit(1);
    }

    printf("Consumidor: Alterando permissões com mprotect...\n");

    // 2. Libera acesso de Leitura e Escrita
    // Nota: Precisa de WRITE também porque o consumidor altera a variável
    // 'out'
    if (mprotect(shared_mem, sizeof(shared_data_t), PROT_READ | PROT_WRITE)
    == -1)

```

```

{
    perror("mprotect");
    exit(1);
}
// -----

sem_t *sem_mutex = sem_open(SEM_MUTEX, 0);
sem_t *sem_empty = sem_open(SEM_EMPTY, 0);
sem_t *sem_full = sem_open(SEM_FULL, 0);

printf("Consumidor iniciado e memória desbloqueada.\n");

while (1)
{
    sem_wait(sem_full);
    sem_wait(sem_mutex);

    int num = shared_mem->buffer[shared_mem->out];
    printf("[Consumidor] Leu: %d\n", num);
    shared_mem->out = (shared_mem->out + 1) % BUFFER_SIZE;

    sem_post(sem_mutex);
    sem_post(sem_empty);

    sleep(2);
}
return 0;
}

```